

Capítulo 5

Interpolación y Aproximación Polinomial

Consideremos la situación en que se conocen los valores de una función en algunos puntos. Esos puntos pueden provenir de mediciones experimentales y por lo tanto podrían contener errores. Se desea hallar una función *sencilla* que coincida en esos puntos, o que la gráfica pase cerca.

Consideremos el caso en que queremos calcular la integral de una función sobre la que no conocemos la *primitiva*, por ejemplo

$$\int_0^1 e^{-x^2} dx.$$

En este caso, lo que se hace, es aproximar la función por funciones sencillas que se sepan integrar, y luego se integran esas funciones sencillas.

Para los matemáticos, las funciones más sencillas son los polinomios. Algunas razones por las que se consideran sencillas son las siguientes:

- (1) Son fáciles de representar, pues basta un número finito de coeficientes. En Octave, un vector $\mathbf{p}$ de n componentes representa el siguiente polinomio de grado $n - 1$:

$$p(x) = \mathbf{p}_1 x^{n-1} + \mathbf{p}_2 x^{n-2} + \cdots + \mathbf{p}_{n-1} x + \mathbf{p}_n.$$

Así, $\mathbf{p} = [6 \ -3 \ 0 \ 1]$ representa el polinomio

$$p(x) = 6x^3 - 3x^2 + 0x + 1 = 6x^3 - 3x^2 + 1$$

Para evaluar un polinomio en un cierto x se utiliza la función **polyval**. Así, para graficar el polinomio dado en Octave, sobre el intervalo $[-3, 3]$, hacemos:

```
>> p = [6 -3 0 1];  
>> x = linspace(-3,3,300);  
>> y = polyval(p,x);  
>> plot(x,y)
```

Resumiendo, para Octave, $\mathbf{p}$ es un vector de 4 componentes. Cuando lo evaluamos con `polyval`, lo interpreta como un polinomio de grado 3. Pero para que lo interprete de esa manera, hay que evaluarlo con `polyval`. Se ruega al lector entender lo que ocurre al hacer las siguientes líneas:

```
>> p(3)
ans = 0
>> polyval(p,3)
ans = 136
>> p(0)
error: subscript indices must be either positive integers less than
2^31 or logicals
>> polyval(p,0)
ans = 1
>> p(4)
ans = 1
>> polyval(p,4)
ans = 337
>> p(0.25)
error: subscript indices must be either positive integers less than
2^31 or logicals
>> polyval(p,0.25)
ans = 0.90625
```

- (2) Son fáciles de integrar, y la integral de un polinomio es otro polinomio:

$$\int 6x^3 - 3x^2 + 0x + 1 \, dx = 6\frac{x^4}{4} - 3\frac{x^3}{3} + 0\frac{x^2}{2} + 1x + c = \frac{6}{4}x^4 - \frac{3}{3}x^3 + \frac{0}{2}x^2 + 1x + c$$

para cualquier constante c . En general

$$\int \mathbf{p}_1 x^{n-1} + \mathbf{p}_2 x^{n-2} + \cdots + \mathbf{p}_{n-1} x + \mathbf{p}_n \, dx = \frac{\mathbf{p}_1}{n} x^n + \frac{\mathbf{p}_2}{n-1} x^{n-1} + \cdots + \frac{\mathbf{p}_{n-1}}{2} x^2 + \mathbf{p}_n x + c.$$

- (3) Son fáciles de derivar, y la derivada de un polinomio es otro polinomio:

$$\frac{d}{dx} (6x^3 - 3x^2 + 0x + 1) = 6 \cdot 3x^2 - 3 \cdot 2x + 0,$$

En general

$$\begin{aligned} \frac{d}{dx} (\mathbf{p}_1 x^{n-1} + \mathbf{p}_2 x^{n-2} + \cdots + \mathbf{p}_{n-1} x + \mathbf{p}_n) \\ = (n-1)\mathbf{p}_1 x^{n-2} + (n-2)\mathbf{p}_2 x^{n-3} + \cdots + 3\mathbf{p}_{n-3} x^2 + 2\mathbf{p}_{n-2} x + \mathbf{p}_{n-1}. \end{aligned}$$

5.1. Interpolación polinomial

Dados dos puntos en el plano, pasa por ellos una única recta.

Si los dos puntos tienen diferente abscisa, entonces hay una única recta de ecuación

$$y = ax + b$$

cuya gráfica pasa por los puntos.

Si tengo tres puntos con diferente abscisa, por ejemplo

$$(1, 8) \quad (-1, 9) \quad (2, 15),$$

veremos que pasa por ellos una única parábola de ecuación

$$y = ax^2 + bx + c.$$

Veamos cómo se calcula dicha parábola. Para que la misma pase por los puntos debe cumplir

$$\begin{array}{lll} (1, 8) \text{ está sobre la parábola} & \Longleftrightarrow & 8 = a1^2 + b1 + c \\ (-1, 9) \text{ está sobre la parábola} & \Longleftrightarrow & 9 = a(-1)^2 + b(-1) + c \\ (2, 15) \text{ está sobre la parábola} & \Longleftrightarrow & 15 = a2^2 + b2 + c. \end{array}$$

Nos queda entonces un sistema de 3 ecuaciones con 3 incógnitas (las incógnitas son a , b , c):

$$\begin{aligned} 1^2a + 1b + c &= 8 \\ (-1)^2a + (-1)b + c &= 9 \\ 2^2a + 2b + c &= 15. \end{aligned}$$

En forma matricial, el sistema se escribe

$$\begin{bmatrix} 1^2 & 1 & 1 \\ (-1)^2 & (-1) & 1 \\ 2^2 & 2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 8 \\ 9 \\ 15 \end{bmatrix}$$

Notamos que el lado derecho de la ecuación es el vector de ordenadas de los puntos (coordenadas y) y las columnas de la matriz son potencias de las abscisas de los puntos (coordenadas x). Para resolverlo en Octave hacemos lo siguiente:

```
>> X = [1 -1 2]';
>> Y = [8 9 15]';
>> matriz = [X.^2 X ones(size(X))]
matriz =

     1     1     1
     1    -1     1
     4     2     1

>> p = matriz\Y
p =

     2.50000
    -0.50000
     6.00000
```

Así, $a = 2.5$, $b = -0.5$ y $c = 6$. Para graficar el polinomio en el intervalo $[-2, 3]$ y los puntos dados hacemos lo siguiente:

```
>> xx = linspace(-2,3,200);
>> yy = polyval(p,xx);
>> plot(xx,yy,X,Y,'*')
```

Ahora nos preguntamos qué ocurre cuando tenemos n puntos. La respuesta está en el siguiente teorema.

Teorema 5.1. *Dados n puntos*

$$(x_1, y_1) \quad (x_2, y_2) \quad \dots \quad (x_n, y_n)$$

con abscisas todas distintas, existe un único polinomio p de grado $n - 1$ cuya gráfica pasa por los puntos, es decir

$$\begin{array}{l} p(x_1) = y_1 \\ p(x_2) = y_2 \\ \vdots \\ p(x_n) = y_n \end{array} \iff p(x_i) = y_i, \quad i = 1, 2, \dots, n.$$

Para demostrar este teorema recordamos que el Teorema Fundamental del Álgebra dice:

Un polinomio de grado $n - 1$ tiene exactamente $n - 1$ ceros en el conjunto de los números complejos.

Sin entrar en detalles sobre lo que son los números complejos, deducimos de ese enunciado que

Un polinomio de grado $n - 1$ tiene a lo sumo $n - 1$ ceros reales.

Esto implica a su vez que:

Si $p(x) = p_1x^{n-1} + p_2x^{n-2} + \dots + p_{n-1}x + p_n$ cumple

$$p(x_i) = 0, \quad i = 1, 2, \dots, n,$$

con todos los x_i distintos, es decir, tiene n ceros, entonces

$$p_1 = p_2 = \dots = p_{n-1} = p_n = 0.$$

Ahora sí, estamos en condiciones de demostrar el teorema.

Demostración del Teorema 5.1. Queremos hallar el polinomio, así que proponemos

$$p(x) = p_1x^{n-1} + p_2x^{n-2} + \dots + p_{n-1}x + p_n.$$

El mismo debe cumplir las siguiente n ecuaciones

$$\begin{aligned} p(x_1) &= y_1 \\ p(x_2) &= y_2 \\ &\vdots \\ p(x_n) &= y_n. \end{aligned}$$

Es decir

$$\begin{aligned} p_1 x_1^{n-1} + p_2 x_1^{n-2} + \cdots + p_{n-1} x_1 + p_n &= y_1 \\ p_1 x_2^{n-1} + p_2 x_2^{n-2} + \cdots + p_{n-1} x_2 + p_n &= y_2 \\ &\vdots \\ p_1 x_n^{n-1} + p_2 x_n^{n-2} + \cdots + p_{n-1} x_n + p_n &= y_n. \end{aligned}$$

Recordando que las abscisas $x_1, x_2, \dots, x_n$ son conocidas y las ordenadas $y_1, y_2, \dots, y_n$ también, las incógnitas son los coeficientes $p_1, p_2, \dots, p_n$ del polinomio. Para encontrarlos debemos resolver ese sistema de ecuaciones que en forma matricial se escribe así:

$$\begin{bmatrix} x_1^{n-1} & x_1^{n-2} & \cdots & x_1 & 1 \\ x_2^{n-1} & x_2^{n-2} & \cdots & x_2 & 1 \\ \vdots & \vdots & & \vdots & \vdots \\ x_n^{n-1} & x_n^{n-2} & \cdots & x_n & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{n-1} \\ p_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix}. \quad (5.1)$$

Observamos nuevamente que el lado derecho del sistema de ecuaciones es el vector de ordenadas de los puntos (coordenadas y) y las columnas de la matriz son potencias de las abscisas de los puntos (coordenadas x). Esta matriz se conoce como matriz de Vandermonde correspondiente a $x_1, x_2, \dots, x_n$.

Este sistema de ecuaciones tiene solución única si la matriz es invertible, lo cual puede verificarse demostrando que el problema homogéneo tiene como única solución a la trivial. Plantear el sistema homogéneo

$$\begin{bmatrix} x_1^{n-1} & x_1^{n-2} & \cdots & x_1 & 1 \\ x_2^{n-1} & x_2^{n-2} & \cdots & x_2 & 1 \\ \vdots & \vdots & & \vdots & \vdots \\ x_n^{n-1} & x_n^{n-2} & \cdots & x_n & 1 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{n-1} \\ p_n \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

equivale a buscar el polinomio que satisface $p(x_i) = 0$, $i = 1, 2, \dots, n$. Por lo que hemos dicho antes de comenzar la demostración, esto implica que $p_1 = p_2 = \cdots = p_{n-1} = p_n = 0$, es decir, la única solución es la trivial.

Por lo tanto, el sistema (5.1) tiene solución única, o lo que es lo mismo, existe un único polinomio cuya gráfica pasa por los n puntos dados. ■

5.1.1. Interpolación por polinomios de Lagrange

En la demostración anterior podemos observar que resolver el sistema 5.1 implica obtener los coeficientes del polinomio interpolante. Para n grande, esta forma de construir $p(x)$ no es práctica, ya que involucra resolver un sistema de n ecuaciones con una matriz llena, y en la mayoría de los casos, las potencias crecientes de los x_i generan una matriz mal condicionada (por lo tanto coeficientes poco confiables por acumulación de errores de redondeo).

Existen varias formas de obtener la expresión del polinomio interpolante que, como se dijo, es único, sin la necesidad de resolver el sistema.

Una forma muy usada de construir polinomios interpolantes es utilizando los *polinomios de Lagrange*. En este caso el polinomio se escribe:

$$p(x) = \sum_{k=1}^n y_k \phi_k(x)$$

donde los $\phi_k(x)$ son polinomios de grado n . Los coeficientes en este caso son los y_k , datos del problema, y la construcción de los polinomios $\phi_k(x)$ es muy sencilla e intuitiva. Se introduce a modo de ejemplo, el caso lineal:

Ejemplo 1. Sea quiere hallar el polinomio $p(x)$ que pase por dos puntos: (x_1, y_1) y (x_2, y_2) , donde $y_1 = f(x_1)$ y $y_2 = f(x_2)$ para alguna $f(x)$ que se quiere aproximar con dicho polinomio. Lo vamos a escribir de la siguiente forma (ver Fig. 5.1):

$$p(x) = \frac{x - x_2}{x_1 - x_2} y_1 + \frac{x - x_1}{x_2 - x_1} y_2 = L_{1,1}(x) y_1 + L_{1,2}(x) y_2$$

donde se ha definido

$$L_{1,1}(x) = \frac{x - x_2}{x_1 - x_2} \quad L_{1,2}(x) = \frac{x - x_1}{x_2 - x_1}$$

Se puede observar que $L_1(x_1) = 1$ y $L_1(x_2) = 0$ y que $L_2(x_1) = 0$ y $L_2(x_2) = 1$.

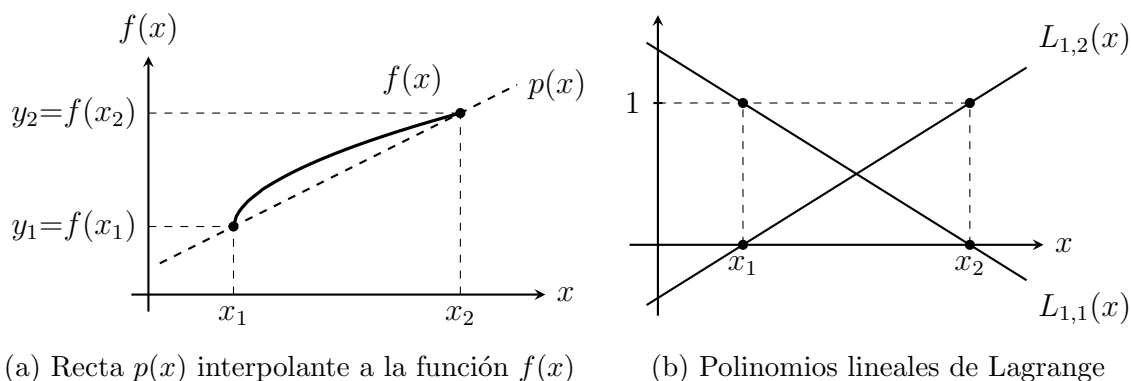


Figura 5.1: Interpretación gráfica del ejemplo.

Esto puede generalizarse, y para un polinomio que pase por n puntos $x_1, x_2, \dots, x_n$ de la siguiente manera:

$$p_{n-1}(x) = L_{n-1,1}(x) y_1 + L_{n-1,2}(x) y_2 + \dots + L_{n-1,n}(x) y_n$$

donde se construirán los $L_{n-1,k}(x)$ de modo que $L_{n-1,k}(x_k) = 1$ y $L_{n-1,k}(x_j) = 0$ para $k \neq j$.

La forma de construir estos polinomios será:

$$L_{n-1,k}(x) = \frac{(x - x_1)(x - x_2) \dots (x - x_{k-1})(x - x_{k+1}) \dots (x - x_n)}{(x_k - x_1)(x_k - x_2) \dots (x_k - x_{k-1})(x_k - x_{k+1}) \dots (x_k - x_n)}$$

Este polinomio $L_{n-1,k}(x)$ así definido es un polinomio de grado $n - 1$ que se denomina *polinomio de Lagrange*, y en notación con símbolo de productoria puede escribirse:

$$L_{n-1,k}(x) = \prod_{\substack{i=1; \\ i \neq k}}^n \frac{(x - x_i)}{(x_k - x_i)}$$

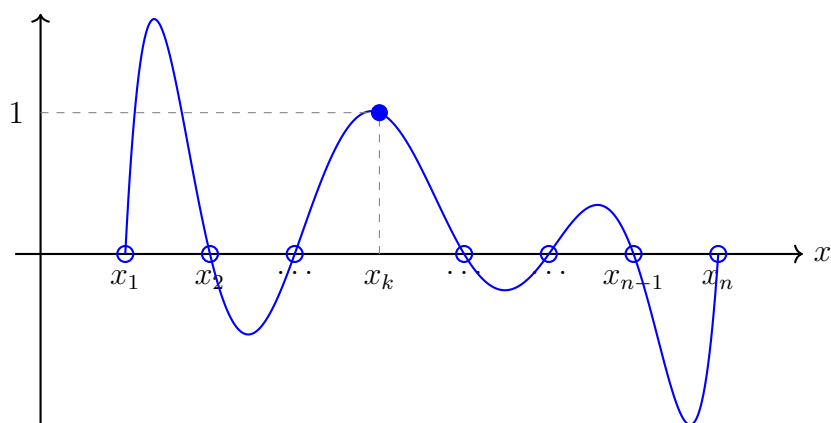


Figura 5.2: Esquema del polinomio de Lagrange $L_{n-1,k}(x)$.

Polinomio interpolador en Octave

En Octave, para hallar el polinomio de grado $n - 1$ que pasa por n puntos, se utiliza la función `polyfit`. Por ejemplo, para hallar el polinomio de grado 4 cuya gráfica pasa por

$$(1, 2) \quad (2, -1) \quad (4.5, 3) \quad (5.5, 0) \quad (10, 3.3)$$

hacemos así:

```
>> X = [1 2 4.5 5.5 10]' ;
>> Y = [2 -1 3 0 3.3]' ;
>> p = polyfit(X,Y,4)
p =
```

```
0.092585    -1.787726    10.949413   -24.722922    17.468651
```

```
>> xx = linspace(0,11,200);
>> yy = polyval(p,xx);
>> plot(xx,yy,X,Y,'*')
```

Esas líneas también incluyen instrucciones para graficar el polinomio en el intervalo $[0, 11]$.

5.1.2. Interpolación de funciones

Volvemos ahora a la idea del polinomio interpolador, es decir, el polinomio cuya gráfica pasa exactamente por una lista de puntos. Pero consideramos ahora que esos puntos coinciden con puntos de la gráfica $y = f(x)$ de una función f dada.

Hagamos un poco de matemática experimental. Para ello, a continuación incluimos una función `pruebapolyfit` que dada una función, un intervalo y un número n de puntos calcula el polinomio interpolante en n puntos equidistantes. La idea es tratar de *medir el error* entre la función y el polinomio hallado.

```
function p = pruebapolyfit(f,a,b,n)
% function p = pruebapolyfit(f,a,b,n)
%
%   Se interpola la funcion f en n
% puntos equiespaciados en el intervalo [a,b].
% En la figura 1 se grafica la funcion f y el polinomio.
% En la figura 2 se grafica el error.
% Ambos graficos se realizan sobre el intervalo [a,b].
%
% La funcion retorna el polinomio p en formato OCTAVE.

% Tomamos n puntos equiespaciados en [a,b]
X=linspace(a,b,n);
% Evaluamos f en esos puntos
Y = f(X);
% Obtenemos el polinomio interpolante
p = polyfit(X,Y,n-1);

% Para graficar tomamos muchos puntos en el intervalo [a,b]
xx = linspace(a,b,200);
% Evaluamos la funcion y el polinomio en todos esos puntos
fxx = f(xx); pxx = polyval(p,xx);
error = abs(fxx-pxx);
maximo_error = max(error)

% Ahora si, graficamos
figure(1)
plot(xx,fxx,'LineWidth',2,xx,pxx,'LineWidth',2,X,Y,'*')
legend('funcion original','polinomio interpolante')
set(gca,'FontSize',18)
```



```
figure(2)
plot(xx,error,'LineWidth',2)
grid, title('Error')
set(gca,'FontSize',18)
```

Ejemplo 2. Veamos qué ocurre cuando interpolamos la función $f(x) = \sin(x)$ sobre el intervalo $[0, 7]$. Para entender mejor se recomienda que baje la función `pruebapolyfit` y realice estas pruebas usted mismo.

```
>> f = @(x) sin(x);
>> pruebapolyfit(f,0,7,2);
maximo_error = 1.4466
>> pruebapolyfit(f,0,7,3);
maximo_error = 1.3319
>> pruebapolyfit(f,0,7,4);
maximo_error = 0.57219
>> pruebapolyfit(f,0,7,5);
maximo_error = 0.31131
>> pruebapolyfit(f,0,7,6);
maximo_error = 0.087421
>> pruebapolyfit(f,0,7,7);
maximo_error = 0.040263
>> pruebapolyfit(f,0,7,8);
maximo_error = 0.0078669
>> pruebapolyfit(f,0,7,9);
maximo_error = 0.0031601
>> pruebapolyfit(f,0,7,10);
maximo_error = 4.6673e-04
```

Vemos en este ejemplo que el error tiende a cero a medida que aumentamos el número n de puntos.

Ejemplo 3. Consideremos ahora la función $f(x) = e^{-x^2}$ sobre el intervalo $[-1, 3]$.

```
>> f = @(x) exp(-x.^2);
>> pruebapolyfit(f,-1,3,2);
maximo_error = 0.72617
>> pruebapolyfit(f,-1,3,3);
maximo_error = 0.58613
>> pruebapolyfit(f,-1,3,4);
maximo_error = 0.20698
>> pruebapolyfit(f,-1,3,5);
maximo_error = 0.23692
>> pruebapolyfit(f,-1,3,6);
maximo_error = 0.084354
>> pruebapolyfit(f,-1,3,7);
```

```

maximo_error = 0.065621
>> pruebapolyfit(f,-1,3,8);
maximo_error = 0.052012
>> pruebapolyfit(f,-1,3,9);
maximo_error = 0.023666
>> pruebapolyfit(f,-1,3,10);
maximo_error = 0.019472

```

Aquí también parece ser que el error tiende a cero, aunque da la impresión de que esto ocurre más lentamente que en el ejemplo anterior.

Veamos un último ejemplo:

Ejemplo 4. Consideremos la función $f(x) = \frac{1}{1+x^2}$ sobre el intervalo $[-5, 5]$.

```

>> f = @(x) 1./(1+x.^2);
>> pruebapolyfit(f,-5,5,2);
maximo_error = 0.96091
>> pruebapolyfit(f,-5,5,3);
maximo_error = 0.64622
>> pruebapolyfit(f,-5,5,4);
maximo_error = 0.70639
>> pruebapolyfit(f,-5,5,5);
maximo_error = 0.43821
>> pruebapolyfit(f,-5,5,6);
maximo_error = 0.43211
>> pruebapolyfit(f,-5,5,7);
maximo_error = 0.61655
>> pruebapolyfit(f,-5,5,8);
maximo_error = 0.24684
>> pruebapolyfit(f,-5,5,9);
maximo_error = 1.0451
>> pruebapolyfit(f,-5,5,10);
maximo_error = 0.30010

```

Aquí hay algo raro. Veamos qué ocurre si seguimos.

```

>> pruebapolyfit(f,-5,5,11);
maximo_error = 1.9156
>> pruebapolyfit(f,-5,5,12);
maximo_error = 0.55418
>> pruebapolyfit(f,-5,5,13);
maximo_error = 3.6554
>> pruebapolyfit(f,-5,5,14);
maximo_error = 1.0639
>> pruebapolyfit(f,-5,5,15);
maximo_error = 7.1908
>> pruebapolyfit(f,-5,5,16);

```

```

maximo_error = 2.0979
>> pruebapolyfit(f,-5,5,17);
maximo_error = 14.329

```

Aquí se ve que la cosa no anda tan bien. El error no parece tender a cero...

Veamos un poco de teoría. El teorema del error de interpolación dice lo siguiente.

Teorema 5.2. Sea f una función definida en el intervalo $[a, b]$. Si $x_1, \dots, x_n$ son todos puntos distintos en el intervalo $[a, b]$, entonces existe un único polinomio $p(x)$ de grado menor o igual a $n - 1$ cuya gráfica pasa por los puntos $(x_i, f(x_i))$ para $i = 0, 1, \dots, n$. Es decir, existe un único polinomio $p(x) = p_1 x^{n-1} + \dots + p_{n-1} x + p_n$ tal que

$$p(x_i) = f(x_i), \quad i = 1, 2, \dots, n.$$

Además, si f es C^n en el intervalo $[a, b]$, se cumple que para cada $x \in [a, b]$

$$|f(x) - p(x)| \leq \frac{M_n}{n!} \prod_{i=1}^n |x - x_i|, \quad \text{donde } M_n = \max_{\xi \in [a, b]} |f^n(\xi)|.$$

Esta cota del error de aproximación sale de la expresión

Corolario 5.3. Si no se sabe dónde están los puntos x_i , entonces, para cada $x \in [a, b]$ sólo sabemos que $|x - x_i| \leq |b - a|$. Del teorema anterior concluimos que:

$$\max_{x \in [a, b]} |f(x) - p(x)| \leq \frac{M_n}{n!} |b - a|^n.$$

Corolario 5.4. Si los puntos x_i están **equiespaciados** y $x_1 = a$, $x_n = b$, es decir

$$x_i = a + (i - 1)\Delta_x, \quad \text{con } \Delta_x = \frac{b - a}{n - 1},$$

entonces, resulta (pensemos que $x \in [x_1, x_2]$)

$$|x - x_1| |x - x_2| \dots |x - x_n| \leq \Delta_x \Delta_x 2\Delta_x 3\Delta_x \dots (n-1)\Delta_x = (n-1)! \Delta_x^n = (n-1)! \frac{|b - a|^n}{(n-1)^n}.$$

Luego, del teorema anterior obtenemos

$$\max_{x \in [a, b]} |f(x) - p(x)| \leq \underbrace{\frac{M_n}{n} \frac{|b - a|^n}{(n-1)^n}}_{\text{cota del error}}.$$

Vale la pena observar que en ambos casos, la cota del error tiene un factor M_n que depende de la derivada de orden n de la función que queremos aproximar, y otro factor, $\frac{|b - a|^n}{n!}$ en un caso y $\frac{|b - a|^n}{n(n-1)^n}$ en el otro. Tanto $\frac{|b - a|^n}{n!}$ como $\frac{|b - a|^n}{n(n-1)^n}$ tienden a cero cuando n tiende a infinito.

En el ejemplo 2, $M_n = 1$ para todo n , así que

$$M_n \frac{|b - a|^n}{n(n-1)^n} = 1 \frac{|b - a|^n}{n(n-1)^n} \rightarrow 0, \quad \text{cuando } n \rightarrow \infty.$$

En el ejemplo 3, resulta más pesado calcular derivadas de todos los órdenes de e^{-x^2} . Cabe aclarar que $M_n \rightarrow \infty$ cuando $n \rightarrow \infty$, pero aún así,

$$M_n \frac{|b-a|^n}{n(n-1)^n} \rightarrow 0, \quad \text{cuando } n \rightarrow \infty.$$

Sin embargo, el valor de M_n puede tender a infinito tan rápido que los términos completos

$$M_n \frac{|b-a|^n}{n!} \quad \text{y} \quad M_n \frac{|b-a|^n}{n(n-1)^n}$$

tiendan a infinito. Este es el caso del ejemplo 4. En ese caso, no podemos asegurar que la aproximación sea cada vez mejor a medida que tomemos más puntos de interpolación y polinomios de grado mayor.

Para asegurar convergencia de la aproximación, se utiliza la interpolación polinomial a trozos.

5.2. Interpolación por *splines* cúbicos

Vimos en la sección anterior que la cota del error para un polinomio p que interpola a una función f en n puntos equiespaciados sobre el intervalo $[a, b]$ es:

$$|f(x) - p(x)| \leq M_n \frac{|b-a|^n}{n(n-1)^n},$$

con $M_n = \max_{\xi \in [a,b]} |f^n(\xi)|$. También vimos que M_n puede ser muy grande para n grande. En este caso, no sirve de mucho aumentar excesivamente el número de puntos de interpolación.

¿Qué se puede hacer para asegurar que el error tiende a cero?

Una cosa que se puede hacer es utilizar interpolación polinomial a trozos.

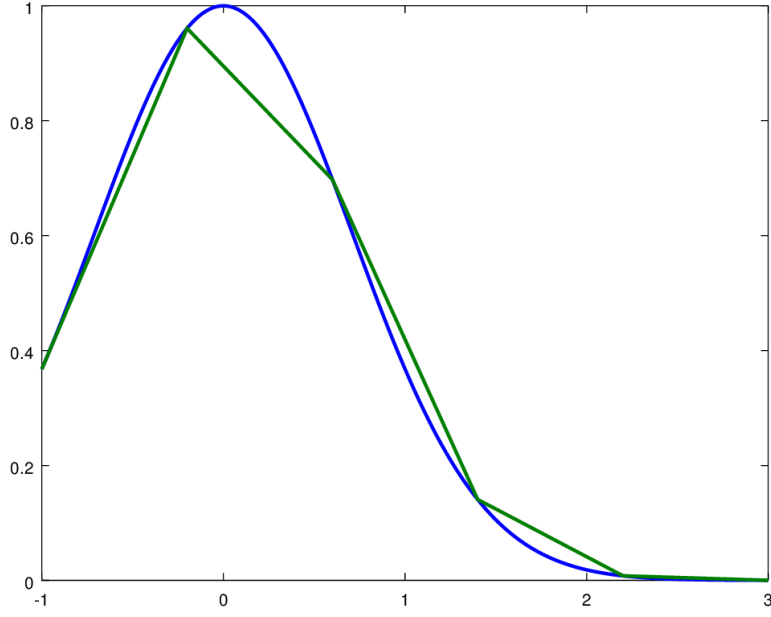
5.2.1. Interpolación polinomial a trozos

La idea es fijar el grado polinomial $n-1$ e interpolar con ese grado (fijo) sobre L sub-intervalos. Más precisamente. Consideremos un intervalo $[a, b]$ y un número entero positivo L . Subdividimos el intervalo $[a, b]$ en L sub-intervalos de longitud $h = \frac{|b-a|}{L}$. Llamamos $y_1, y_2, \dots, y_{L+1}$ a los extremos de esos sub-intervalos, es decir

$$y_1 = a, \quad y_2 = a+h, \quad y_3 = a+2h, \quad \dots \quad y_L = a+(L-1)h = b-h, \quad y_{L+1} = a+Lh = b.$$

Luego, interpolamos en cada sub-intervalo con un polinomio de grado $n-1$.

En el gráfico a continuación se muestra un ejemplo para $n=2$. Es decir, en cada sub-intervalo $[y_i, y_{i+1}]$ se interpola con una función lineal (polinomio de grado $n-1=1$) que coincide en los extremos de ese sub-intervalo.



En el caso $n = 3$ consideramos, en cada sub-intervalo, el polinomio de grado 2 que interpola en los extremos y en el punto medio de ese sub-intervalo. Así, se puede realizar para cualquier grado polinomial.

Luego, si fijamos n (no demasiado grande), y miramos el error en un sub-intervalo, resulta, por el teorema del error de interpolación (Teorema 5.2), lo siguiente:

$$|f(x) - p(x)| \leq \frac{M_n}{n(n-1)^n} \left(\frac{|b-a|}{L} \right)^n.$$

Notemos que hemos cambiado $|b-a|$ por $\frac{|b-a|}{L}$, ya que estamos considerando un sub-intervalo, que tiene longitud $h = \frac{|b-a|}{L}$. Resumimos esta observación en el siguiente teorema:

Teorema 5.5. *Sea f es una función C^n sobre el intervalo $[a, b]$. Si p es el polinomio a trozos de grado $\leq n-1$ sobre una partición de tamaño $h = \frac{|b-a|}{L}$, que interpola a f en $(L+1) + (n-2)L$ puntos equiespaciados, entonces*

$$|f(x) - p(x)| \leq \frac{M_n}{n(n-1)^n} \left(\frac{|b-a|}{L} \right)^n = \frac{M_n}{n(n-1)^n} h^n,$$

para todo punto $x \in [a, b]$, con $M_n = \max_{\xi \in [a, b]} |f^n(\xi)|$.

Observemos que n está fijo, y que podemos tomar L grande. Si hacemos tender L a infinito, resulta que

$$|f(x) - p(x)| \leq \frac{M_n}{n(n-1)^n} \left(\frac{|b-a|}{L} \right)^n = \frac{M_n |b-a|^n}{n(n-1)^n} \frac{1}{L^n} \rightarrow 0, \quad \text{cuando } L \rightarrow \infty.$$

5.2.2. *Splines* o trazadores cúbicos

Una desventaja de la aproximación descrita anteriormente es que probablemente no existe diferenciabilidad en los extremos de los subintervalos, rompiendo la “suavidad”. A menudo, a partir de las condiciones físicas, se requiere conservar suavidad, por lo que la función de aproximación debe ser continuamente diferenciable.

La aproximación polinomial por tramos más común usa polinomios cúbicos entre cada par sucesivo de nodos y recibe el nombre de **interpolación de spline cúbico**, o **trazador cúbico**. Un polinomio cúbico general implica cuatro coeficientes constantes a determinar, por lo que existe bastante flexibilidad en el procedimiento de spline cúbico para garantizar que el interpolante no sólo es continuamente diferenciable en el intervalo, sino también tiene una segunda derivada continua.

Observación 17. Es importante destacar aquí que la obtención del polinomio trazador no implica que las derivadas en los nodos coincidan con información de los datos. Por ejemplo, si se desea interpolar una función con un trazador cúbico, este último tendrá hasta la segunda derivada continua, pero no será necesariamente coincidente con dichas derivadas de la función (no se utiliza para su construcción las derivadas de la función original).

Definición 5.1. Dada una función f definida en el intervalo $[a, b]$ y un conjunto de $n + 1$ nodos $a = x_1 < x_2 < \dots < x_n < x_{n+1} = b$, un **interpolante trazador cúbico** (*spline*) de f es una función S definida a partir de las siguientes condiciones:

- i) $S(x)$ es un polinomio cúbico por tramos, denotando $S_j(x)$ en el intervalo $[x_j, x_{j+1}]$ para cada $j = 1, \dots, n$
- ii) $S_j(x_j) = f(x_j)$ para cada $j = 1, 2, \dots, n$
- iii) $S_{j-1}(x_j) = S_j(x_j)$ para cada $j = 2, 3, \dots, n$
- iv) $S'_{j-1}(x_j) = S'_j(x_j)$ para cada $j = 2, 3, \dots, n$
- v) $S''_{j-1}(x_j) = S''_j(x_j)$ para cada $j = 2, 3, \dots, n$
- vi) $S_n(x_{n+1}) = f(x_{n+1})$
- vii) Se cumple uno de las siguientes condiciones:
 - a) $S''(x_1) = S''(x_{n+1}) = 0$ (**frontera natural** o libre).
 - b) $S'(x_1) = f'(x_1)$ y $S'(x_{n+1}) = f'(x_{n+1})$ (**frontera sujeta**).

De la definición, las condiciones **iii)**, **iv)** y **v)** aseguran la continuidad de $S(x)$ hasta la segunda derivada. Si se cuentan las ecuaciones que se obtienen en las diferentes condiciones, se tienen $4n - 2$ ecuaciones, más las dos que proporciona alguna de las opciones de la última condición. Esto suma las $4n$ ecuaciones que se necesitan para determinar los $4n$ coeficientes (4 por cada uno de los n polinomios $S_j(x)$ que conforman el polinomio por tramos $S(x)$).

5.2.3. Construcción del trazador

Para la obtención de los coeficientes, conviene expresar cada S_j de la siguiente manera:

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3, \quad j = 1, 2, \dots, n \quad (5.2)$$

De la definición 5.1, la condición i) determina los coeficientes a_i :

$$S_j(x_j) = a_j = f(x_j) \quad j = 1, 2 \dots n$$

Si se define $a_{n+1} = f(x_{n+1})$ y el tamaño de cada sub-intervalo $h_j = x_{j+1} - x_j$, de la condición iii) de la definición 5.1 se obtienen las siguientes ecuaciones:

$$a_{j-1} + b_{j-1}h_{j-1} + c_{j-1}h_{j-1}^2 + d_{j-1}h_{j-1}^3 = a_j \quad j = 2, 3 \dots n \quad (5.3)$$

Si se deriva (5.2), se obtiene

$$S'_i(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2 \quad (5.4)$$

Definiendo ahora $b_{n+1} = S'(x_{n+1})$, la condición iv) nos proporciona las siguientes ecuaciones:

$$b_{j-1} + 2c_{j-1}h_{j-1} + 3d_{j-1}h_{j-1}^2 = b_j \quad j = 2, 3, \dots, n \quad (5.5)$$

Se define ahora $c_{n+1} = S''(x_{n+1})/2$ y se deriva nuevamente (5.2)

$$S''_i(x) = 2c_i + 6d_i(x - x_i) \quad (5.6)$$

Al aplicar la condición v), se obtiene

$$c_{j-1} + 3d_{j-1}h_{j-1} = c_j \quad j = 2, 3 \dots, n \quad (5.7)$$

Despejando d_{j-1} de la ecuación (5.7):

$$d_{j-1} = (c_j - c_{j-1})/3h_{j-1} \quad (A)$$

Ya se obtuvo una expresión para los coeficientes d_j en función de los c_j . Lo que se realizará a continuación es reemplazar esta ecuación (A) en las ecuaciones (5.3) y (5.5) con el objetivo de seguir simplificando las ecuaciones:

Entonces, reemplazando (A) en (5.3)

$$a_j = a_{j-1} + b_{j-1}h_{j-1} + (2c_{j-1} + c_j)h_{j-1}^2/3 \quad (5.8)$$

despejamos b_{j-1} :

$$b_{j-1} = (a_j - a_{j-1})/h_{j-1} - (2c_{j-1} - c_j)h_{j-1}/3 \quad (B)$$

Ahora, reemplazando (A) en (5.5)

$$b_j = b_{j-1} + h_{j-1}(c_{j-1} + c_j) \quad (5.9)$$

y usando (B), se obtiene finalmente:

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_jc_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}) \quad (5.10)$$

para $j = 2, 3, \dots, n$. Las ecuaciones (5.10) representan un sistema lineal de $n - 1$ ecuaciones, en c_j . Si se aplican las condiciones vi) de la definición 5.1, se agregarían 2 nuevas ecuaciones al sistema (5.10) que involucren a c_1 y c_{n+1} . Se puede demostrar que el sistema final tiene solución única. Por lo tanto, determinados $c_1, c_2, \dots, c_{n+1}$, con la ecuación (A) se determinan los coeficientes d_j , haciendo lo propio con los b_j mediante la ecuación (B).

Spline cúbico natural

Tomando en cuenta la expresión (5.6), la condición de **frontera natural** implica que $c_1 = 0$ y $c_{n+1} = 0$. Luego, el sistema final con estas dos ecuaciones y (5.10), se representa como $Ac = b$, donde $c = (c_1, c_2, \dots, c_n, c_{n+1})^t$ es el vector de incógnitas, con las primeras n componentes los coeficientes c_j del spline natural. La matriz A queda conformada como:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & 0 & 0 & \dots & 0 & 0 \\ 0 & h_2 & 2(h_2 + h_3) & h_3 & 0 & \dots & 0 & 0 \\ 0 & 0 & h_3 & 2(h_3 + h_4) & h_4 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & h_{n-1} & 0 \\ 0 & 0 & 0 & 0 & 0 & h_{n-1} & 2(h_{n-1} + h_n) & h_n \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 & 1 \end{bmatrix}$$

y el vector b del lado derecho

$$\mathbf{b} = \begin{bmatrix} 0 \\ \frac{3}{h_2}(a_3 - a_2) - \frac{3}{h_1}(a_2 - a_1) \\ \frac{3}{h_3}(a_4 - a_3) - \frac{3}{h_2}(a_3 - a_2) \\ \vdots \\ \frac{3}{h_n}(a_{n+1} - a_n) - \frac{3}{h_{n-1}}(a_n - a_{n-1}) \\ 0 \end{bmatrix}$$

Este sistema así definido tiene solución única, ya que la matriz A es estrictamente diagonal dominante. En este sentido, podemos afirmar el siguiente resultado:

Teorema 5.6. *Si f se define en $a = x_1 < x_2 < \dots < x_n < x_{n+1} = b$, entonces f tiene un spline cúbico natural único S , que interpola en los nodos $x_1, x_2, \dots, x_n, x_{n+1}$.*

Spline cúbico sujeto

La condición de **frontera sujeta** se aplica sobre la ecuación (5.4), generando un nuevo sistema de características similares al spline natural. En efecto, como $f'(a) = S'(a) = S'(x_1) = b_1$, se tiene en (B) con $j = 2$:

$$f'(a) = (a_2 - a_1)/h_1 - (2c_1 + c_2)h_1/3 \quad (5.11)$$

reordenando la expresión, se genera la siguiente ecuación:

$$2h_1c_1 + h_1c_2 = 3(a_2 - a_1)/h_1 - 3f'(a) \quad (5.12)$$

De manera similar se aplica la otra condición en la ecuación (5.9) para $j = n + 1$

$$f'(b) = b_{n+1} = b_n + h_n(c_n + c_{n+1}) \quad (5.13)$$

y usando la ecuación (B) para $j = n$ y reordenando, se obtiene

$$h_nc_n + 2h_nc_{n+1} = 3f'(b) - 3(a_{n+1} - a_n)/h_n \quad (5.14)$$

Finalmente, juntando las ecuaciones (5.10) con (5.12) y (5.14), se obtiene, al igual que en el caso natural, un sistema lineal $Ac = b$, con:

$$\mathbf{A} = \begin{bmatrix} 2h_1 & h_1 & 0 & 0 & 0 & \dots & 0 & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & 0 & 0 & \dots & 0 & 0 \\ 0 & h_2 & 2(h_2 + h_3) & h_3 & 0 & \dots & 0 & 0 \\ 0 & 0 & h_3 & 2(h_3 + h_4) & h_4 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & h_{n-1} & 0 \\ 0 & 0 & 0 & 0 & 0 & h_{n-1} & 2(h_{n-1} + h_n) & h_n \\ 0 & 0 & 0 & 0 & 0 & \dots & h_n & 2h_n \end{bmatrix}$$

y el lado derecho

$$\mathbf{b} = \begin{bmatrix} \frac{3}{h_1}(a_2 - a_1) - 3f'(a) \\ \frac{3}{h_2}(a_3 - a_2) - \frac{3}{h_1}(a_2 - a_1) \\ \frac{3}{h_3}(a_4 - a_3) - \frac{3}{h_2}(a_3 - a_2) \\ \vdots \\ \frac{3}{h_n}(a_{n+1} - a_n) - \frac{3}{h_{n-1}}(a_n - a_{n-1}) \\ 3f'(b) - \frac{3}{h_n}(a_{n+1} - a_n) \end{bmatrix}$$

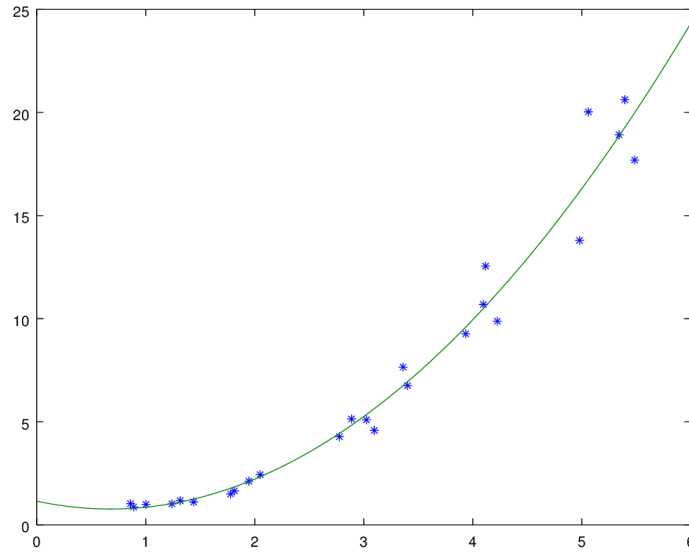
Nuevamente, se observa que la matriz es estrictamente diagonal dominante, garantizando el siguiente teorema:

Teorema 5.7. *Si f se define en $a = x_1 < x_2 < \dots < x_n < x_{n+1} = b$, y es diferenciable en a y b , entonces f tiene un spline cúbico sujeto único S , que interpola en los nodos $x_1, x_2, \dots, x_n, x_{n+1}$.*

5.3. Aproximación por cuadrados mínimos

En la sección anterior vimos cómo calcular un polinomio *interpolador* de una lista de puntos. Es decir, buscábamos el polinomio cuya gráfica pasa exactamente por todos los puntos dados. Volveremos a esto en la sección siguiente, lo que será útil para obtener fórmulas de integración numérica en el capítulo próximo.

Ahora nos enfocaremos en el cálculo de un polinomio de grado bajo que pase *cerca* de una lista dada de puntos.



Consideremos el caso en que tenemos n puntos

$$(x_1, y_1) \quad (x_2, y_2) \quad \dots \quad (x_n, y_n)$$

con n grande, como en la gráfica, y buscamos un polinomio de grado menor o igual a 2, cuya gráfica pase lo más cerca posible de esos puntos. Debemos definir en qué sentido el polinomio pasa cerca de los puntos. La manera más usual es lo que se conoce como la aproximación por cuadrados mínimos, que consiste en lo siguiente. Queremos hallar el polinomio $p(x)$ que minimice la suma de la distancia entre $p(x_i)$ y y_i al cuadrado, es decir, que minimice la siguiente expresión:

$$(p(x_1) - y_1)^2 + (p(x_2) - y_2)^2 + \dots + (p(x_n) - y_n)^2 = \sum_{i=1}^n (p(x_i) - y_i)^2.$$

Geométricamente, estamos mirando la distancia *vertical* entre el punto *dato* (x_i, y_i) y el punto de la gráfica con la misma abscisa $(x_i, p(x_i))$. Lo que se quiere minimizar es la suma de esas distancias al cuadrado. En notación más compacta, queremos minimizar la norma euclídea o magnitud de este vector:

$$\begin{bmatrix} p(x_1) \\ p(x_2) \\ \vdots \\ p(x_n) \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} p_1 x_1^2 + p_2 x_1 + p_3 \\ p_1 x_2^2 + p_2 x_2 + p_3 \\ \vdots \\ p_1 x_n^2 + p_2 x_n + p_3 \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Observamos que

$$\begin{bmatrix} p_1 x_1^2 + p_2 x_1 + p_3 \\ p_1 x_2^2 + p_2 x_2 + p_3 \\ \vdots \\ p_1 x_n^2 + p_2 x_n + p_3 \end{bmatrix} = \underbrace{\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix}}_M \underbrace{\begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix}}_{\underline{p}}$$

y planteamos el siguiente problema.

Hallar el vector $\underline{p}$ que minimice la norma de $M\underline{p} - \underline{y}$, es decir, que minimice:

$$\|M\underline{p} - \underline{y}\| = \left(\sum_{i=1}^n [(M\underline{p})_i - y_i]^2 \right)^{1/2}$$

Para resolver este problema, recordemos que $M\underline{p}$ es una combinación lineal de las columnas de M :

$$M\underline{p} = p_1 \text{col}_1(M) + p_2 \text{col}_2(M) + p_3 \text{col}_3(M),$$

y por lo tanto, el problema consiste en encontrar el elemento del *espacio columna* de M que está lo más cerca posible de $\underline{y}$.

Recordando un poco de geometría, vemos que $M\underline{p}$ es el elemento de dicho espacio más cercano a $\underline{y}$ si y sólo si el *error* $\underline{y} - M\underline{p}$ es perpendicular al espacio generado por las columnas de M . Es decir, si $\underline{y} - M\underline{p}$ es perpendicular a cada columna de M . Más precisamente, si se cumple que

$$\begin{aligned} \text{col}_1(M) \cdot (\underline{y} - M\underline{p}) &= 0 \\ \text{col}_2(M) \cdot (\underline{y} - M\underline{p}) &= 0 \\ \text{col}_3(M) \cdot (\underline{y} - M\underline{p}) &= 0. \end{aligned}$$

Recordando que si u, v son vectores columna, entonces $u \cdot v = u^T v$ resulta que $M\underline{p}$ debe cumplir

$$\begin{aligned} \text{col}_1(M)^T (\underline{y} - M\underline{p}) &= 0 \\ \text{col}_2(M)^T (\underline{y} - M\underline{p}) &= 0 \\ \text{col}_3(M)^T (\underline{y} - M\underline{p}) &= 0. \end{aligned}$$

En notación más compacta

$$\begin{bmatrix} \text{col}_1(M)^T \\ \text{col}_2(M)^T \\ \text{col}_3(M)^T \end{bmatrix} (\underline{y} - M\underline{p}) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \iff M^T (\underline{y} - M\underline{p}) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

A su vez, esta última expresión, equivale a

$$M^T M \underline{p} = M^T \underline{y}.$$

Es decir, el vector $\underline{p}$ (de tres componentes) debe ser la solución al sistema de ecuaciones

$$(M^T M) \underline{p} = M^T \underline{y}$$

En Octave, si $\mathbf{X}, \mathbf{Y}$ son vectores que tienen las abscisas y las ordenadas de los puntos, el cálculo del polinomio p se haría de la siguiente manera:

(1) Armado de la matriz M ($\mathbf{X}, \mathbf{Y}$ deben ser vectores columna)

```
>> M = [ X.^2  X  ones(size(X)) ]
```

(2) Armamos la matriz del sistema a resolver, la llamamos A

```
>> A = M'*M
```

(3) Armamos el lado derecho del sistema a resolver, lo llamamos b

```
>> b = M'*Y
```

(4) Hallamos los coeficientes del polinomio que aproxima en el sentido de mínimos cuadrados:

```
>> p = A \ b
```

(5) Ahora graficamos el polinomio y los puntos en un mismo gráfico. Supongamos para ello que estamos en el intervalo $[0, 6]$:

```
>> xx = linspace(0,6,200);  
>> yy = polyval(p,xx);  
>> plot(X,Y, '*',xx,yy)
```

Los pasos (1) a (4) de lo recién explicado pueden resumirse en Octave usando `polyfit`:

```
>> p = polyfit(X,Y,2)
```

Observación 18. Recalcamos que igualmente es importante entender cómo se resuelve un sistema en el sentido de mínimos cuadrados, pues hay ocasiones en que se quieren aproximar puntos por funciones de otra forma, por ejemplo: $p(x) = a3^x + b2^x + c$. Allí hay que reproducir los pasos, utilizando como matriz M la siguiente:

```
>> M = [ 3.^X 2.^X ones(size(X)) ]
```

Observación 19. El estudio del error cometido por el polinomio hallado y la función real que modela el comportamiento de los puntos, que han sido medidos con un cierto error aleatorio, escapa al alcance de este curso, y corresponde al área Estadística.